



مهلت: ۱۱ آبان

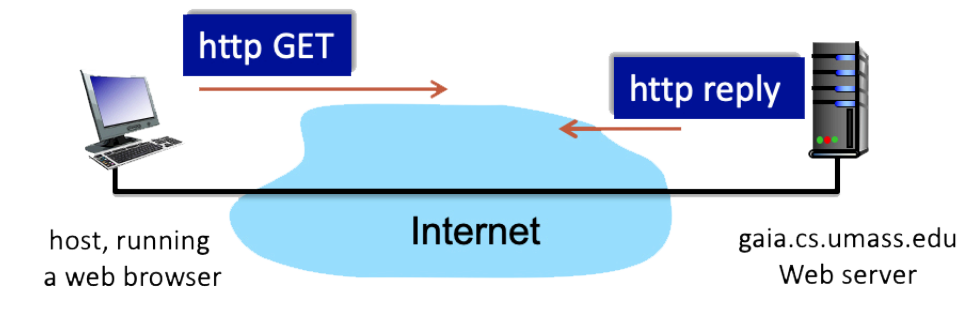
شبکه‌های کامپیوتری
Fall 2025



تمرین دوم

دانشجویانی که رقم آخر شماره دانشجویی آنها زوج است باید به سوالات (1, 2, 5, 7, 9) پاسخ دهند و دانشجویانی که رقم آخر شماره دانشجویی آنها فرد است باید به سوالات (3, 4, 6, 7, 8) پاسخ دهند. توجه داشته باشید که انجام سوال ۱۰ (آزمایشگاه وب سرور) برای تمامی دانشجویان الزامی است.

۱. در شکل زیر سرور در حال ارسال یک پیام HTTP RESPONSE به کلاینت است. فرض کنید پیام ارسالی از سرور به کلاینت به صورت زیر باشد:



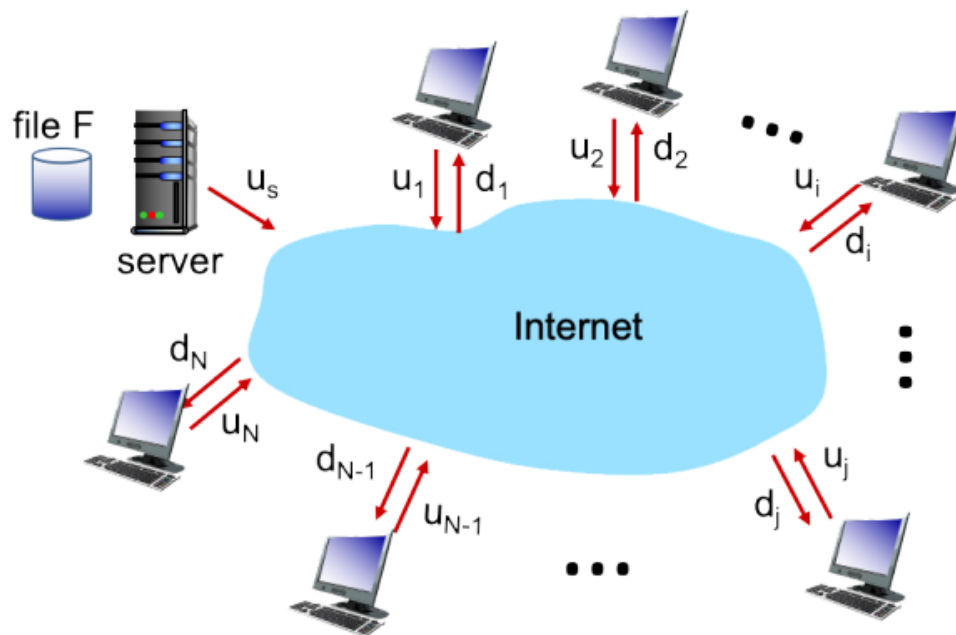
```
HTTP/1.1 200 OK
Date: Sat, 18 Oct 2025 18:43:30 +0000
Server: Apache/2.2.3 (CentOS)
Last-Modified: Sat, 18 Oct 2025 18:56:50 +0000
ETag: 17dc6-a5c-bf716880.
Content-Length: 53993
Keep-Alive: timeout=43, max=70
Connection: Keep-alive
Content-type: image/html
```

- (آ) آیا پیام پاسخ از HTTP 1.0 استفاده می‌کند یا HTTP 1.1؟
- (ب) آیا سرور توانسته است سند را با موفقیت ارسال کند؟ دلیل خود را توضیح دهید.
- (ج) اندازه‌ی سند چند بایت است؟
- (د) نوع فایلی که سرور در پاسخ ارسال می‌کند چیست؟

۲. در این مسئله، شما زمان موردنیاز برای توزیع یک file را که در ابتدا روی یک server قرار دارد، به clients از طریق client-server یا peer-to-peer مقایسه خواهید کرد. مسئله این است که یک file با اندازه $F = 5 \text{ Gbits}$ به هر یک از ۸ peer توزیع شود. فرض کنید server دارای نرخ upload برابر با $u = 87 \text{ Mbps}$ است. هر یک از ۸ peer دارای نرخ‌های upload و download به شرح زیر هستند:

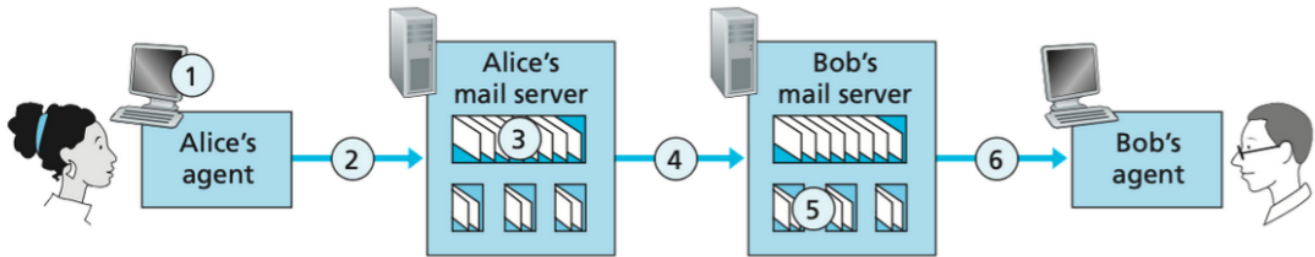
Table 1: upload and download rates for all 8 peers

peer	$d_i()$	$u_i()$
1	10	23
2	12	24
3	13	10
4	40	25
5	30	17
6	13	19
7	34	30
8	37	21



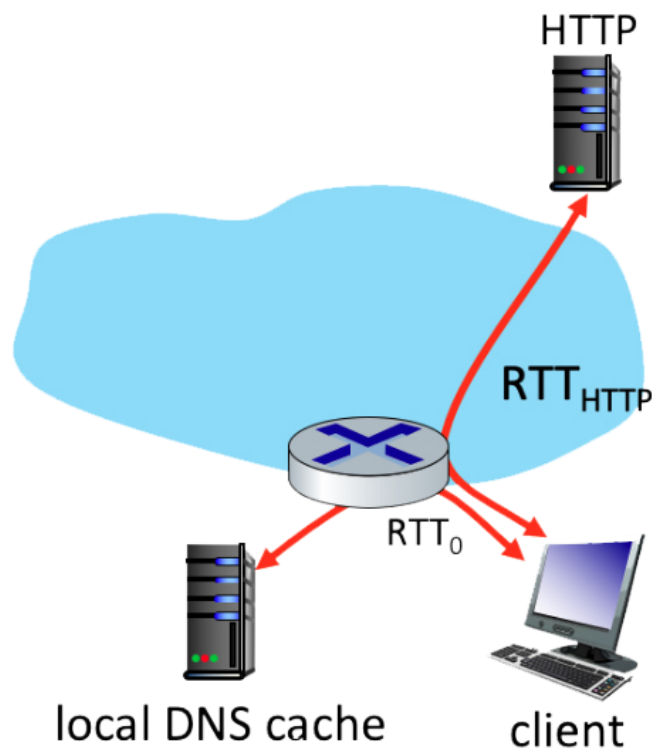
حداقل زمانی که لازم است تا این file از central server به ۸ peer با استفاده از client-server model توزیع شود، چقدر است؟

۳. به سناریوی زیر توجه کنید که در آن Alice یک ای‌میل برای Bob ارسال می‌کند. فرض کنید هر دو، یعنی Alice و Bob، از پروتکل POP3 استفاده می‌کنند. در آن صورت به هر یک از پرسش‌های زیر پاسخ دهید.

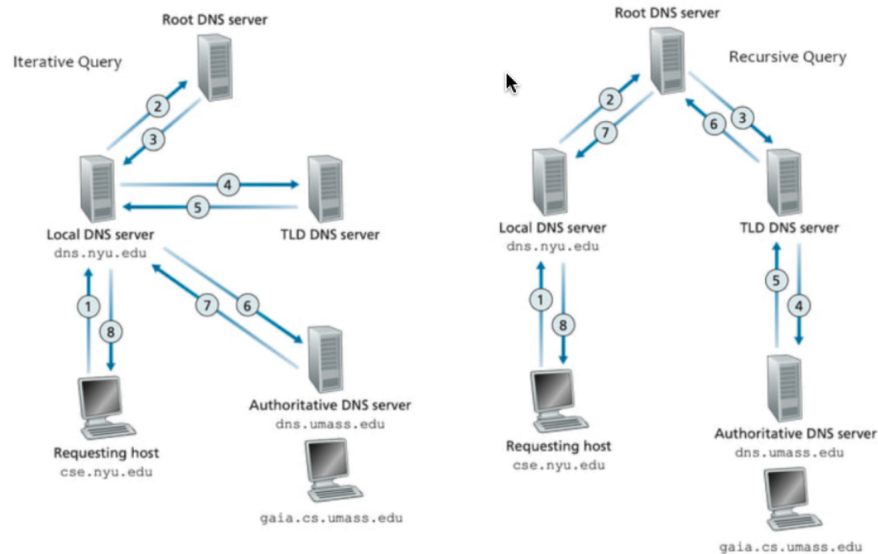


- * در نقطه ۲، از چه پروتکلی استفاده می‌شود؟
- * در نقطه ۴، از چه پروتکلی استفاده می‌شود؟
- * در نقطه ۶، از چه پروتکلی استفاده می‌شود؟
- * آیا پروتکل SMTP از TCP استفاده می‌کند یا از UDP؟
- * آیا SMTP یک پروتکل push است یا pull؟
- * آیا POP3 یک پروتکل push است یا pull؟
- * SMTP از چه شماره پورتهای استفاده می‌کند؟
- * POP3 از چه شماره پورتهای استفاده می‌کند؟

۴. فرض کنید درون مرورگر وب خود روی یک لینک کلیک می‌کنید تا یک صفحه وب دریافت کنید. آدرس IP مربوط به URL در میزبان محلی کش نشده است، بنابراین یک DNS lookup لازم است تا آدرس IP به دست آید. فرض کنید تنها یک DNS server، یعنی local DNS cache، با یک RTT تاخیر برابر با $RTT_0 = 1 \text{ msec}$ مورد پرس‌وجو قرار می‌گیرد. در ابتدا فرض کنید صفحه‌ی وب مربوط به لینک دقیقاً شامل یک شی است که از مقدار کمی متن HTML تشکیل شده است. فرض کنید RTT بین میزبان محلی و Web server شامل شی برابر $RTT_{HTTP} = 40 \text{ msec}$ است. حال با فرض زمان انتقال صفر برای شی HTML، چه مقدار زمان (برحسب میلی ثانیه) از زمانی که کاربر روی لینک کلیک می‌کند تا زمانی که کلاینت شی را دریافت می‌کند، سپری می‌شود؟



۵. فرض کنید کاربری می‌خواهد به سایت `gaia.cs.umass.edu` مراجعه کند، اما مرورگر او آدرس IP این وبسایت را نمی‌داند. در این مثال، با توجه به نوع درخواست‌های DNS به سوالات داده شده پاسخ دهید.



درخواست‌های Iterative

★ بین مراحل ۱ و ۲، سرور DNS محلی ابتدا کجا را بررسی می‌کند؟ پاسخ را از بین User، DNS Local، DNS Root، DNS TLD، یا DNS Authoritative انتخاب کنید.

★ بین مراحل ۲ و ۳، اگر سرور DNS ریشه آدرس IP مورد نظر را نداشته باشد، پاسخ به کجا ارجاع داده می‌شود؟ پاسخ را از بین DNS Local، DNS Root، DNS TLD، یا DNS Authoritative انتخاب کنید.

★ بین مراحل ۴ و ۵، اگر سرور DNS سطح دامنه (TLD) آدرس IP مورد نظر را نداشته باشد، پاسخ به کجا ارجاع داده می‌شود؟ پاسخ را از بین DNS Local، DNS Root، DNS TLD، یا DNS Authoritative انتخاب کنید.

★ بین مراحل ۶ و ۷، سرور DNS معتبر (Authoritative) با آدرس IP مورد نظر پاسخ می‌دهد. چه نوع رکورد DNS بازگردانده می‌شود؟

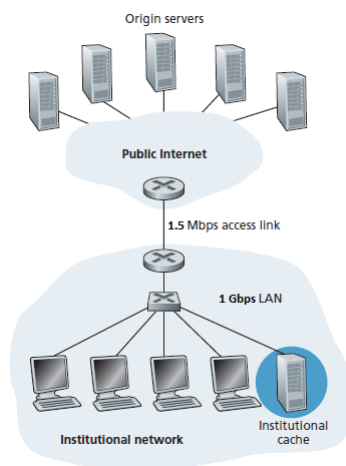
درخواست‌های Recursive

- * بین مراحل ۱ و ۲، سرور DNS محلی ابتدا کجا را بررسی می‌کند؟ پاسخ را از بین User، DNS Local، DNS Root، DNS TLD یا DNS Authoritative انتخاب کنید.
- * بین مراحل ۲ و ۳، سرور DNS ریشه درخواست را به کجا ارسال می‌کند؟ پاسخ را از بین DNS Local، DNS Root، DNS TLD یا DNS Authoritative انتخاب کنید.
- * بین مراحل ۴ و ۵، سرور DNS معتبر پاسخ را به کجا ارسال می‌کند؟ پاسخ را از بین DNS Local، DNS Root، DNS TLD یا DNS Authoritative انتخاب کنید.
- * در مراحل ۶ تا ۸، پاسخ در مسیر معکوس تا رسیدن به کاربر ارسال می‌شود. چه نوع رکورد DNS بازگردانده می‌شود؟

* کدام نوع درخواست به عنوان روش بهتر در نظر گرفته می‌شود: Iterative یا Recursive؟

۶. فرض کنید ۱۰ کلاینت با استفاده از پروتکل FTP به طور همزمان در حال دریافت فایل‌های با حجم زیاد از یک فایل سرور هستند و لینک گلوگاه لینک متصل به سرور است. اگر یکی از کلاینت‌ها از یک برنامه مدیریت دانلود (Download Manager) استفاده کند که به طور همزمان ۹ اتصال TCP باز می‌کند در آن صورت سرعت دانلود این کلاینت نسبت به قبل (یک اتصال TCP) چندبرابر خواهد شد؟

۷. در شبکه زیر کاربران حاضر در LAN در حال دریافت فایل از سرورهای واقع در اینترنت می‌باشند. فرض کنید متوسط زمان دریافت یک فایل هنگامی که institutional cache فعال نیست ۲ ثانیه است. پس از فعال شدن institutional cache متوسط زمان دریافت یک فایل ۱.۷ ثانیه بوده و نرخ اصابت (cache hit) در institutional cache چهل درصد است. متوسط زمان صرف شده برای هر فایل در LAN چند ثانیه است؟



۸. فرض کنید فایلی به حجم 1 Gbits را می‌خواهیم بین N نظیر (peer) توزیع کنیم. نرخ آپلود سرور را 20 Mbps، نرخ دانلود هر نظیر را 10 Mbps و نرخ آپلود هر نظیر را 1 Mbps در نظر بگیرید.

✱ حداقل زمان توزیع را برای $N = 10$ و $N = 1000$ و $N = 10000$ در هر دو توزیع P2P و Client-Server به دست آورید.

✱ اعداد بدست آمده در قسمت قبل را مقایسه کنید. چگونه تغییر می‌کنند؟

۹. فرض کنید شما یک صفحه وب را که شامل یک سند و پنج عکس است را درخواست می‌کنید. اندازه سند ۱ کیلوبایت، و حجم هر تصویر ۵۰ کیلوبایت، سرعت دانلود ۱ مگابیت در ثانیه و RTT هم 100 ns است. در حالت‌های زیر چه مدت طول می‌کشد تا کل محتوای سایت را دریافت کنیم. (فرض کنید نیازی به DNS query نیست و زمان جابه‌جایی هدرها و دیگر سربارها در پیام‌های HTTP ناچیز است.)

✱ non-persistent HTTP با connection های سری

✱ non-persistent HTTP با دو connection موازی

✱ persistent HTTP با یک connection

۱۰. آزمایش وب سرور

در این آزمایشگاه، شما مبانی برنامه‌نویسی سوکت برای اتصالات TCP در پایتون را یاد خواهید گرفت: چگونه یک سوکت ایجاد کنید، آن را به یک آدرس و پورت مشخص متصل (bind) کنید، و همچنین چگونه یک بسته HTTP را ارسال و دریافت نمایید. شما همچنین با مبانی فرمت هدر HTTP آشنا خواهید شد.

شما یک وب سرور توسعه خواهید داد که در هر زمان یک درخواست HTTP را مدیریت می‌کند. وب سرور شما باید درخواست HTTP را بپذیرد و تجزیه (parse) کند، فایل درخواست شده را از سیستم فایل سرور دریافت کند، یک پیام پاسخ HTTP شامل فایل درخواست شده به همراه خطوط هدر ایجاد کند، و سپس پاسخ را مستقیماً به کلاینت ارسال نماید. اگر فایل درخواست شده در سرور موجود نباشد، سرور باید یک پیام HTTP "404 Not Found" به کلاینت بازگرداند.

کد

در زیر ساختار کد (skeleton code) برای وب سرور را خواهید یافت. شما باید این کد را تکمیل کنید. مکان‌هایی که باید کد را پر کنید با `#Fill - in - start` و `#Fill - in - end` مشخص شده‌اند. هر مکان ممکن است به یک یا چند خط کد نیاز داشته باشد.

اجرای سرور

یک فایل HTML (مثلاً `HelloWorld.html`) را در همان دایرکتوری که سرور در آن قرار دارد، قرار دهید. برنامه سرور را اجرا کنید. آدرس IP میزبانی که سرور را اجرا می‌کند، مشخص کنید (مثلاً `128.238.251.26`). از یک میزبان دیگر، یک مرورگر باز کرده و URL مربوطه را وارد کنید. برای مثال:

`http://128.238.251.26:6789/HelloWorld.html`

`HelloWorld.html` نام فایلی است که شما در دایرکتوری سرور قرار داده‌اید. همچنین به استفاده از شماره پورت بعد از دو نقطه توجه کنید. شما باید این شماره پورت را با هر پورتی که در کد سرور استفاده

کرده‌اید، جایگزین کنید. در مثال بالا، ما از شماره پورت 6789 استفاده کرده‌ایم. سپس مرورگر باید محتویات HelloWorld.html را نمایش دهد. اگر 6789 را حذف کنید، مرورگر پورت 80 را در نظر می‌گیرد و شما صفحه وب را از سرور دریافت خواهید کرد تنها در صورتی که سرور شما روی پورت 80 در حال گوش دادن باشد.

سپس سعی کنید فایلی را که در سرور موجود نیست، درخواست کنید. شما باید یک پیام "404 Not Found" دریافت کنید.

آنچه باید تحویل دهید

شما باید کد کامل سرور را به همراه اسکرین‌شات‌هایی از مرورگر کلاینت خود تحویل دهید که تأیید می‌کند محتویات فایل HTML را واقعاً از سرور دریافت کرده‌اید.

ساختار کد پایتون برای وب سرور

```
#import socket module
from socket import *
import sys # In order to terminate the program

serverSocket = socket(AF_INET, SOCK_STREAM)
#Prepare a sever socket
#Fill in start
#Fill in end

while True:
    #Establish the connection
    print('Ready to serve...')
    connectionSocket, addr = #Fill in start
    #Fill in end
    try:
        message = #Fill in start
        #Fill in end
        filename = message.split()[1]
        f = open(filename[1:])
```

```

        outputdata = #Fill in start
        #Fill in end

        #Send one HTTP header line into socket
        #Fill in start
        #Fill in end

        #Send the content of the requested file to the client
        for i in range(0, len(outputdata)):
            connectionSocket.send(outputdata[i].encode())
        connectionSocket.send("\r\n".encode())

        connectionSocket.close()
    except IOError:
        #Send response message for file not found
        #Fill in start
        #Fill in end

        #Close client socket
        #Fill in start
        #Fill in end

serverSocket.close()
sys.exit() #Terminate the program after sending the corresponding data

```

تمرین‌های اختیاری

بخش اول

در حال حاضر، وب سرور در هر زمان فقط یک درخواست HTTP را مدیریت می‌کند. یک سرور چند نخ‌ی (multithreaded) پیاده‌سازی کنید که قادر به پاسخگویی همزمان به چندین درخواست باشد. با استفاده از نخ‌ها (threading)، ابتدا یک نخ اصلی ایجاد کنید که در آن سرور اصلاح شده شما روی یک پورت ثابت برای کلاینت‌ها گوش می‌دهد. هنگامی که یک درخواست اتصال TCP از یک کلاینت دریافت می‌کند، اتصال TCP را از طریق یک پورت دیگر برقرار کرده و درخواست کلاینت را در یک نخ جداگانه سرویس می‌دهد. برای هر زوج درخواست/پاسخ، یک اتصال TCP جداگانه در یک نخ جداگانه وجود خواهد داشت.

بخش دوم

به جای استفاده از مرورگر، کلاینت HTTP خود را برای تست سرور بنویسید. کلاینت شما با استفاده از یک اتصال TCP به سرور متصل می‌شود، یک درخواست HTTP به سرور ارسال می‌کند و پاسخ سرور را به عنوان خروجی نمایش می‌دهد. می‌توانید فرض کنید که درخواست HTTP ارسالی از نوع متد GET است. کلاینت باید آرگومان‌های خط فرمان را برای مشخص کردن آدرس IP یا نام میزبان سرور، پورتهای سرور در آن گوش می‌دهد، و مسیری که شیء درخواستی در سرور ذخیره شده است، دریافت کند. فرمت دستور ورودی برای اجرای کلاینت به صورت زیر است:

```
client.py server_host server_port filename
```

نکاتی که باید توجه داشته باشید:

- الف) مهلت ارسال در سربرگ تمرین همچنین در ایلرن درج شده است.
- ب) کلیه تمرینات (غیر از تمرینات سرکلاسی) از طریق ایلرن دریافت می شوند و دیگر شیوه های ارسال تمرین پذیرفته نیست.
- ج) در صورتی که تمرینات در قالب $\text{\LaTeX}$ و تنها در Template تمرینات ارسال شوند، ۵٪ نمره‌ی اضافی به عنوان امتیاز در نظر گرفته خواهد شد. (Template در ایلرن در دسترس است).
- د) فایل تمرینات ارسالی باید شامل فایل‌های مورد نیاز باشد. در صورتی که تمرینات در قالب $\text{\LaTeX}$ تهیه شده باشند، لازم است فایل‌های مورد نیاز جهت اجرای فایل $\text{\LaTeX}$ به همراه فایل PDF نیز ارسال شوند. نام فایل را به صورت زیر انتخاب کنید:

CN#_StudentNumber#_StudentName

- ه) ارسال با تأخیر تمرین‌ها و پروژه‌های درس صرفاً بر اساس قوانین ذکرشده در فصل صفر امکان‌پذیر خواهد بود.